Build Your Own Backend

---

json-server in 30 Minutes

CSCI 39548 — Practical Web Development

Why This Deck Exists

- You've been fetching from `http://localhost:3001/todos` all class
- Today we build that `:3001` ourselves — from a single JSON file
- No Express. No database. No route handlers.
- One JSON file $\rightarrow$ full REST API. Setup time: ~90 seconds.

The Big Idea — JSON File as Database

Real backend	json-server
Express + Prisma + Postgres	One `db.json` file
~200 lines of route handlers	Zero handlers
30 min to scaffold	30 seconds
Production-ready	Dev/teaching only

It's not a real backend. It's a **stand-in** so you can practice the frontend side of fetch/cache without learning a database tonight.

What You'll Have When This Is Done

- A `db.json` file with your own data shape
- An `npm run server` script that starts the API on port 3001
- Auto-generated REST endpoints for every top-level key
- A `db.seed.json` + `npm run reset-db` you can re-run after destructive demos

Step 1: Install json-server

```
npm install --save-dev json-server@0.17.4
```

- `--save-dev` — it's a dev tool, not a production dependency
- `@0.17.4` — **pin this exact version**

Why Pin to 0.17.4

- json-server **v1.x rewrote the entire CLI**
- The `--watch` flag is gone, the file format changed
- Most tutorials online still teach 0.17.x
- If you install "latest" you'll spend an hour debugging why `npm run server` does nothing
- **Pin 0.17.4 and move on**

Sanity Check

```
npx json-server --version
# 0.17.4
```

Step 2: Write db.json

Create `db.json` at the project root:

```
{
  "todos": [
    { "id": 1, "text": "Buy groceries",
      "done": false, "minutes": 30 },
    { "id": 2, "text": "Walk the dog",
      "done": true,  "minutes": 20 },
    { "id": 3, "text": "Read 20 pages",
      "done": false, "minutes": 40 }
  ],
  "users": [
    { "id": 1, "name": "Ada Lovelace",
      "email": "ada@example.com" },
    { "id": 2, "name": "Alan Turing",
      "email": "alan@example.com" }
  ]
}
```

The Two Rules

1. **Top-level keys become endpoints.**
   `"todos"` → `/todos`. `"users"` → `/users`. That's it.

2. **Every row must have an `id`.**
   Numeric or string. json-server uses it for `/todos/:id` lookups and for PATCH/DELETE. No `id` → 404 on detail pages.

Designing Your Own Data Shape

Three questions before you write a line of JSON:

Question	Example
What are your **nouns**? (each becomes a top-level key)	`recipes`, `ingredients`, `users`
What **fields** does each noun have?	`recipe: { id, title, minutes, vegan }`
How do nouns **link**? (foreign-key style)	`ingredient.recipeId` → `recipe.id`

Plain JSON. No schema, no migrations. Want to add a field tomorrow? Add it to the JSON. Done.

Seed Data Tips

- **5–10 rows minimum.** Empty list demos look broken even when they work.
- **Mix the booleans.** Some `done: true`, some `done: false` — so your filters have something to filter.
- **Spread the foreign keys.** Don't assign every todo to user 1.
- **Hand-write the first few ids as 1, 2, 3.** Easy to type in URLs while testing.

Step 3: Add the npm Scripts

In `package.json`, add to `"scripts"`:

```
"scripts": {
  "dev":       "vite",
  "server":    "json-server --watch db.json --port 3001",
  "reset-db": "cp db.seed.json db.json"
}
```

Note: On Windows PowerShell, `cp` doesn't exist. Use `copy db.seed.json db.json` instead, or install `shx` for cross-platform.

Two Scripts, Two Terminals

Terminal	Command	What it runs	Port
A	`npm run server`	json-server (the API)	3001
B	`npm run dev`	Vite (the React app)	5173

- **Two processes. Two tabs.** Don't run them in the same one.
- Label your tabs in your terminal.
- The React app on `:5173` will `fetch` from json-server on `:3001`.

Why Port 3001 (Not 3000)

- Vite defaults to port 3000 in some configs
- If json-server also tries 3000, one process fails silently
- **Force** `--port 3001` in the script — you never have to think about it again

Step 4: The Seed File

The moment you `POST` or `DELETE`, json-server **rewrites** `db.json` on disk. After 30 mutations your demo data is unrecognizable.

```
cp db.json db.seed.json
```

`db.seed.json` is your factory reset.

The Reset Workflow

You ran	Effect
`npm run server` + mutation demos	`db.json` is now mutated
`npm run reset-db`	`db.seed.json` overwrites `db.json`
Restart `npm run server`	Back to factory state

Treat `db.seed.json` as the canonical copy. Edit it when you want to add seed rows, then re-run `npm run reset-db`.

Step 5: Start the Server

```
npm run server
```

You should see:

```
  Resources
  http://localhost:3001/todos
  http://localhost:3001/users
```

Those URLs exist because there are top-level keys in `db.json`. **You wrote zero handlers.**

Verify in the Browser

URL	What you should see
`http://localhost:3001/todos`	JSON array of all todos
`http://localhost:3001/todos/1`	One todo object (id 1)
`http://localhost:3001/todos?done=false`	Filtered — only undone
`http://localhost:3001/todos?assigneeId=1`	All todos for user 1
`http://localhost:3001/users`	JSON array of users

This is the entire backend. Five URLs, raw JSON, no code.

Auto-Generated Endpoints — Reads

Method	URL	What it does
GET	`/todos`	List all
GET	`/todos/1`	One by id
GET	`/todos?done=false`	Filter by field
GET	`/todos?assigneeId=1`	Filter by foreign key
GET	`/todos?_sort=minutes&_order=desc`	Sort
GET	`/todos?_limit=2`	Pagination (first 2)

Auto-Generated Endpoints — Writes

Method	URL	Body	Returns
POST	`/todos`	New row (no `id`)	Created row with generated `id`
PATCH	`/todos/1`	`{ done: true }`	Updated row (partial)
PUT	`/todos/1`	Full row	Updated row (full replace)
DELETE	`/todos/1`	—	{}

POST auto-assigns the `id`**.** Don't send one — json-server picks the next available integer.

The 7 Pitfalls

#	Symptom	Cause	Fix
1	`Failed to fetch`	Forgot to start json-server	Run `npm run server`
2	Output fighting	Ran both in one tab	One process per tab
3	"Address in use"	Something else on 3001	Kill it or change port
4	`/todos/1` $\rightarrow$ 404	Row missing `id`	Every row needs an `id`
5	Edit ignored	Mutation overwrote edit	Edit `db.seed.json` + reset
6	Data unrecognizable	Mutations stacked up	`npm run reset-db`
7	`--watch` unknown	Installed v1.x	Pin `0.17.4`

Connect It to React

With `useEffect`:

```
useEffect(() => {
  fetch("http://localhost:3001/todos")
    .then((r) => r.json())
    .then(setTodos);
}, []);
```

Or with TanStack Query:

```
const { data: todos } = useQuery({
  queryKey: ["todos"],
  queryFn: () => fetch("http://localhost:3001/todos")
    .then((r) => r.json()),
});
```

The Whole Pipeline

```
db.json -> json-server :3001 -> fetch() -> React component
   ^                                            |
   +---------- PATCH /todos/1 -----------------+
```

- File on disk $\rightarrow$ REST API in 30 seconds
- Same `fetch` you'd use against a real backend
- When you swap to Express, **the React code doesn't change**

What You Can Do Now

- Design a `db.json` for your project (3 nouns, 5–10 rows each) ✓
- Wire `npm run server` + `npm run reset-db` scripts ✓
- Copy `db.json` → `db.seed.json` ✓
- Verify all endpoints in the browser before writing any React ✓
- Build your features against your own backend ✓

When to Outgrow json-server

Stay on json-server	Move to Express
Coursework, prototypes	Anything you'd deploy
Solo project, local dev only	Multi-user, auth, real DB
Schema is "just JSON"	You need validation, transactions
You don't care about losing data	The data matters

For this course, json-server is enough through A4. The final project moves to Express + Prisma.